

01

Outline

인사 패키지의 검증 기록

- 사용자는 이름을 입력해 인사말을 받는다. 다른 개발자는 테스트로 같은 동작을 재현한다.
- 입력·기대값을 정하고 자동으로 실행해 실패 원인을 찾는다.
- 실행하지 않은 코드와 서로 간섭하는 테스트를 찾아 고친다.

완성할 산출물

- `hello-open-source` 저장소의 함수 테스트와 명령 입력 테스트를 완성한다.
- 테스트 파일·실행 명령·검사 결과를 README에 함께 남겨 다른 개발자에게 전달한다.

03

명령 입력에서 출력까지

src/hello_open_source/cli.py 전체 코드

- CLI는 Command-Line Interface, 즉 터미널의 명령 입력 접점이다. `sys.argv`의 0번은 명령 이름, 1번부터 사용자 입력이다.
- `import sys`는 인수 목록을, `from ... import greet`는 인사 함수를 가져온다. 이름이 없으면 `world`를 선택한다.
- `print`는 문자열 뒤에 줄바꿈을 출력한다. `return 0`은 정상 종료 값이며 `-> int`는 정수 반환 표기다.

```
import sys
from hello_open_source.greeting import greet

def main() -> int:
    if len(sys.argv) > 1:
        name = sys.argv[1]
    else:
        name = "world"
    print(greet(name))
    return 0
```

04

첫 pytest 테스트

tests/test_greeting.py: 준비 → 실행 → 단언

- `pytest` 는 테스트를 찾아 실행·보고하는 도구다. 한 테스트는 입력·기대값·비교를 묶고, 스위트는 여러 테스트의 집합이다.
- `from ... import greet` 는 공개 함수를 가져온다. `"Hee"` 를 준비해 호출하고 결과를 `actual` 에 담는다.
- `assert` 는 비교가 참이면 통과, 거짓이면 실패다. `test_` 는 발견 규칙, `-> None` 은 반환값이 없다는 형식 표기다.

```
from hello_open_source.greeting import greet
```

```
def test_greet_with_name() -> None:  
    actual = greet("Hee")  
    assert actual == "Hello, Hee!"
```

실제 `"Hello, Hee!" == 기대 "Hello, Hee!"` → 통과
문자열이라는 사실만 검사하면 `"BROKEN"`도 통과할 수 있다.

05

테스트 환경과 첫 실행

같은 Python에 패키지와 도구 설치

- 모든 경로·명령의 기준은 `hello-open-source/` 다. 프로젝트 가상환경을 활성화하고 기존 패키지를 설치한다.
- `python -m pip` 는 현재 환경에 설치한다. `-e .` 는 현재 소스를 편집 가능한 설치로 연결한다.

```
python -m pip install -e .
python -m pip install "pytest>=8,<10" \
    "pytest-cov>=6,<8"
```

```
python -c "import sys; print(sys.executable)"
python -c "import hello_open_source.greeting as g;
print(g.__file__)"
```

확인 예: `/home/student/hello-open-source/.venv/bin/python`
과 `/home/student/hello-open-source/src/hello_open_source/greeting.py`

기본 명령의 자동 발견·실행·보고

- `python -m pytest` 는 현재 Python으로 검사한다. 파일 경로는 검사 대상, `-q` 는 간결한 보고 옵션이다.
- 점과 `1 passed` 는 한 사례의 통과다. `[100%]` 는 진행률이며 코드 실행 비율이 아니다.

```
python -m pytest
python -m pytest tests/test_greeting.py -q
```

```
. [100%]
1 passed
(경과 시간은 실행마다 다름)
```

06

실패 진단과 회귀 검사

기대값이 계약과 다른 경우

- 계약과 실제값은 `Hello, Hee!` 인데 기대값을 `Hi, Hee!` 로 쓴 상황이다. 테스트를 고친다.
- `E` 는 오류 설명, `-` 는 오른쪽 기대값, `+` 는 왼쪽 실제값이다.

```
assert greet("Hee") == "Hi, Hee!"
```

```
E  AssertionError
E    - Hi, Hee!
E    + Hello, Hee!
1 failed
```

구현이 계약과 다른 경우

- 회귀는 변경 때문에 이전 동작이 깨지는 현상이다. 구현에서 `!` 를 지웠다면 구현을 고친다.
- 정확한 기대값의 테스트를 유지한다. 같은 결함이 재등장하면 같은 입력으로 다시 실패한다.

계약: `"Hello, Hee!"`

구현에서 `!` 삭제 → `"Hello, Hee"`
기존 테스트 실행 → 1 failed

구현의 `!` 복구 → `"Hello, Hee!"`
같은 테스트 실행 → 1 passed

07

pytest 발견 규칙과 설정

기본 발견과 선택적 탐색 제한

- 기본은 현재 폴더의 하위를 재귀 탐색한다.
`test_*.py`, `*_test.py` 안의 `test_` 함수를 찾는다.
- `pyproject.toml`의 아래 표·키는 경로 인수 없는 실행의 시작 폴더를 지정한다. 설치 위치 설정이 아니며 필수도 아니다.
- `--collect-only`는 실행 없이 수집한다. `::`는 파일과 함수 식별자를 구분한다.

```
[tool.pytest.ini_options]
testpaths = ["tests"]
```

```
python -m pytest --collect-only -q
tests/test_greeting.py::test_greet_with_name
1 test collected
```

교재 Figure 5.1의 포함 관계

- 왼쪽은 폴더(Directories)→파일(Modules)→클래스(Classes)→함수의 탐색 순서, 오른쪽은 실제 포함 관계다.
- `Test*` 클래스는 선택 사항이다. 기본 수집에서는 `__init__`이 없어야 하며 일반 함수 `greet`는 수집되지 않는다.

Starting in the directories defined by the configured `testpaths` value, pytest will recursively search for tests.

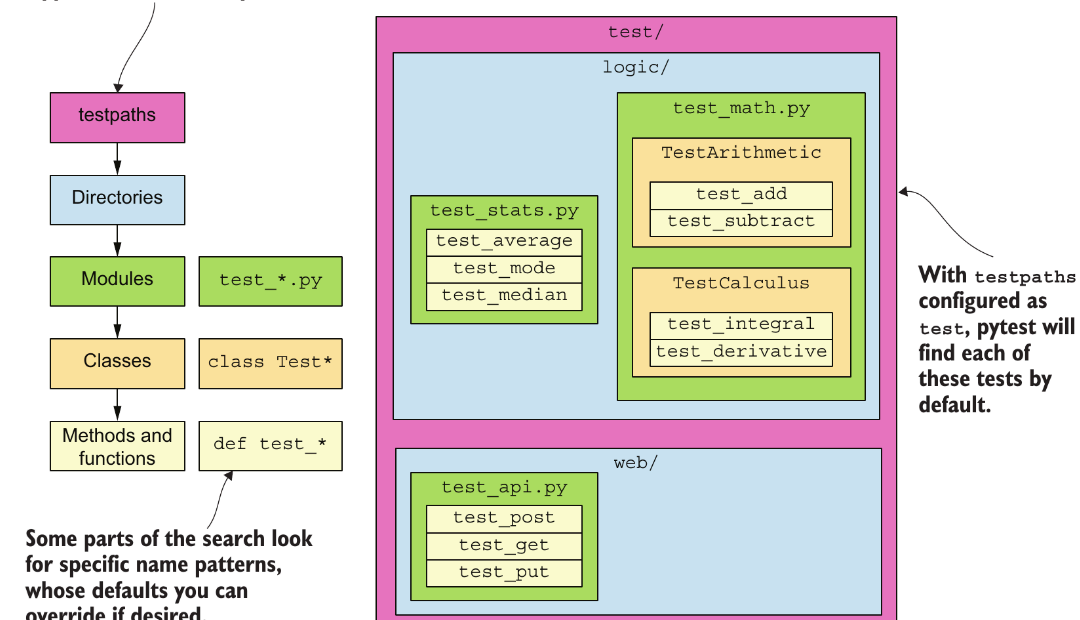


Figure 5.1 pytest discovers the unit tests in a project using recursive pattern matching.

08

pytest fixture

테스트 인자로 받는 준비·복구 도구

- fixture는 pytest가 테스트에 제공하는 데이터나 도구다. 테스트 함수의 인자 이름으로 요청하면 실행 전에 준비한다.
- `monkeypatch` 는 `sys.argv` 를 임시 교체하고 종료 시 복구한다. 실제 셸의 인수나 파일을 영구 수정하지 않는다.
- `capsys` 는 표준 출력·오류를 모은다. `readouterr()` 로 읽은 결과의 `.out` 이 표준 출력 문자열이다.

```
준비: argv를 ["hello-greet", "Hee"]로 임시 교체
실행: main() → 0 반환, "Hello, Hee!\n" 출력
검사: 반환값과 캡처한 문자열 각각 비교
종료: 원래 argv로 자동 복구
```

09

CLI 점점 테스트

hello-open-source/tests/test_cli.py

- `monkeypatch.setattr(sys, "argv", ...)` 는 대상 모듈·속성 이름·새 값을 받는다. 이 목록을 `main` 이 읽는다.
- `main()` 반환값 0과 화면 출력은 서로 다른 관찰값이다. 둘 다 계약과 비교한다.
- `\n` 은 `print` 가 붙인 줄바꿈이다. 아래 한 테스트를 실행하면 `1 passed` 가 나온다.

```
import sys
from hello_open_source.cli import main

def test_main_with_name(monkeypatch, capsys) -> None:
    monkeypatch.setattr(sys, "argv", ["hello-greet", "Hee"])
    result = main()
    assert result == 0
    assert capsys.readouterr().out == "Hello, Hee!\n"
```

매개변수화로 반복 테스트 통합

tests/test_cli.py의 완성 코드

- 기존 테스트를 교체한다. 같은 절차를 복사하는 대신 입력·기대값 두 행을 하나의 함수에 전달한다.

```
import sys
import pytest
from hello_open_source.cli import main

@pytest.mark.parametrize(
    ("argv", "expected"),
    [
        ("hello-greet", "Hee"),
        ("hello-greet", "Hello, world!"),
    ],
    ids=["name", "default"],
)
def test_main(argv, expected, monkeypatch, capsys):
    monkeypatch.setattr(sys, "argv", argv)
    result = main()
    assert result == 0
    assert capsys.readouterr().out == expected
```

두 데이터 행 → 두 독립 검사

- `@pytest.mark.parametrize(...)` 는 바로 아래 함수를 꾸민다. 첫 튜플은 인자 이름, 리스트의 각 튜플은 한 사례다.
- 행의 첫 값은 `argv`, 둘째 값은 `expected` 로 들어간다. `ids` 는 행 순서의 사례 이름이다.
- 준비 도구는 fixture가 제공한다. 기본 실행은 한 행이 실패해도 다음 행을 검사하며 `-v` 는 이름별 결과를 보여 준다.

```
python -m pytest tests/test_cli.py -v
```

```
test_main[name] PASSED
test_main[default] PASSED
```

함수 정의 1개 × 데이터 행 2개
→ 독립 사례 2개
(출력의 파일 경로는 생략)

11

coverage 측정과 보고서

실행한 줄·분기 경로의 측정

- coverage는 실행 여부를 센다. 줄은 주석·빈 줄을 제외한 실행 가능한 줄, 분기는 조건의 가능한 다음 경로다.
- `pytest-cov`는 Coverage.py를 연결하는 확장 모듈이다. 기본 pytest 실행에는 이 측정이 없다.
- `--cov`는 대상, `--cov-branch`는 분기 측정, `term-missing`은 미실행 줄 표시다.

```
python -m pytest \
  --cov=hello_open_source \
  --cov-branch --cov-report=term-missing
```

소스는 읽고 `.coverage`에 결과를 기록한다. 기본 재실행은 이전 측정값을 교체한다.

cli.py에서 한 경로가 빠진 예

- `Stmts / Miss`는 전체·미실행 줄 수, `Branch`는 전체 분기 도착 경로 수다.
- `BrPart`는 실행된 분기 줄에서 빠진 도착 경로 수다. 모든 미실행 분기의 합은 아니다.
- `Missing 9`는 미실행 9번 줄이다. `Cover`는 줄과 분기를 합산하며 결과의 정확성은 단언으로 검사한다.

```
Stmts 8 / Miss 1 / Branch 2 / BrPart 1
Cover 80% / Missing 9
```

```
전체 기회 = 줄 8 + 경로 2 = 10
실행 기회 = 줄 7 + 경로 1 = 8
Cover = 8 / 10 × 100 = 80%
```

12

미실행 경로와 추가할 입력

이름 입력 한 사례만 실행

- `[name]` 은 사례 이름이다. 아래 명령으로 그 행만 실행하면 통과하지만 `cli.py` 의 생략 경로가 빠진다.
- `Missing 9` 는 `name = "world"` 다. 다른 이름을 넣어도 같은 참 경로이므로 입력을 생략해야 한다.
- 거짓 경로의 기대값 `Hello, world!\n` 과 반환값 0까지 단언해야 실행이 동작 검증으로 이어진다.

```
python -m pytest "tests/test_cli.py::test_main[name]" \
  --cov=hello_open_source --cov-branch --cov-report=term-missing
```

```
cli.py: Stmts 8 / Miss 1 / Branch 2 / BrPart 1 / Cover 80% / Missing 9
1 passed
```

```
["hello-greet", "Hee"] → 길이 > 1 참 → name = "Hee"
["hello-greet"]       → 길이 > 1 거짓 → name = "world"
```

13

전체 테스트와 coverage 합산

전체 명령에서 두 경로와 반환값 검사

- `python -m pytest --cov=hello_open_source --cov-branch --cov-report=term-missing` 으로 전체 사례를 실행한다.
- 명령 입력 2개와 함수 반환값 1개가 통과한다. 같은 줄의 반복 실행은 중복 합산하지 않는다.
- **TOTAL** 은 파일별 퍼센트의 단순 평균이 아니다. 전체 줄·분기 기회를 분모와 분자에 각각 합친다.

```
Name                               Stmts Miss Branch BrPart Cover
src/hello_open_source/__init__.py      0     0      0      0  100%
src/hello_open_source/cli.py           8     0      2      0  100%
src/hello_open_source/greeting.py      2     0      0      0  100%
TOTAL                                10     0      2      0  100%
3 passed
```

합산 비교: `greeting 2/2 + cli 8/10 → 10/12 ≈ 83%` (이름만 검사)
 `greeting 2/2 + cli 10/10 → 12/12 = 100%` (두 경로 검사)

14

100% coverage의 두 가지 한계

실행됐지만 단언하지 않은 반환값

- `tests/test_greeting.py`에서 아래 함수는 `greet`를 호출하고도 반환값을 검사하지 않는다.
- 구현이 `"BROKEN"`을 반환해도 `assert True`는 통과한다. 정확한 기대 문자열과 비교해야 한다.

```
def test_only_executes() -> None:
    greet("Hee")
    assert True
```

```
greeting.py: Stmts 2 / Miss 0 / Cover 100%
"Hello, Hee!" 반환 → 1 passed
"BROKEN" 반환      → 1 passed
```

한 줄에 감춰진 다른 입력 선택

- `cli.py`의 이름 선택을 아래 한 줄로 바꾸면 한쪽 값만 선택해도 그 줄이 실행된 것으로 기록된다.
- 줄 간 이동을 재는 도구는 이 조건식의 양쪽을 별도 분기로 세지 않을 수 있다. 이름·생략 두 입력의 계약 검사는 여전히 필요하다.

```
name = sys.argv[1] if len(sys.argv) > 1 else "world"
```

이름만 실행한 한 줄 버전의 실제 측정:

```
cli.py: Stmts 6 / Miss 0 / Branch 0 / Cover 100%
```

여러 줄 if/else 버전에서는 Branch 2.
코드 모양과 의미상의 입력 선택을 구분한다.

15

정상 경로와 입력 영역의 분할

교재 Figure 5.2의 코드 실행 경로

- 파란 Code 상자가 코드다. 초록 점선은 흔한 정상 경로, 자홍 점선은 드문 경계·오류 경로이며 화살표는 진행 방향이다.
- Edge case는 경계 사례, Error handling은 오류 처리다. 분기 측정은 빠진 경로를 찾지만 중요한 입력을 대신 선택하지 않는다.

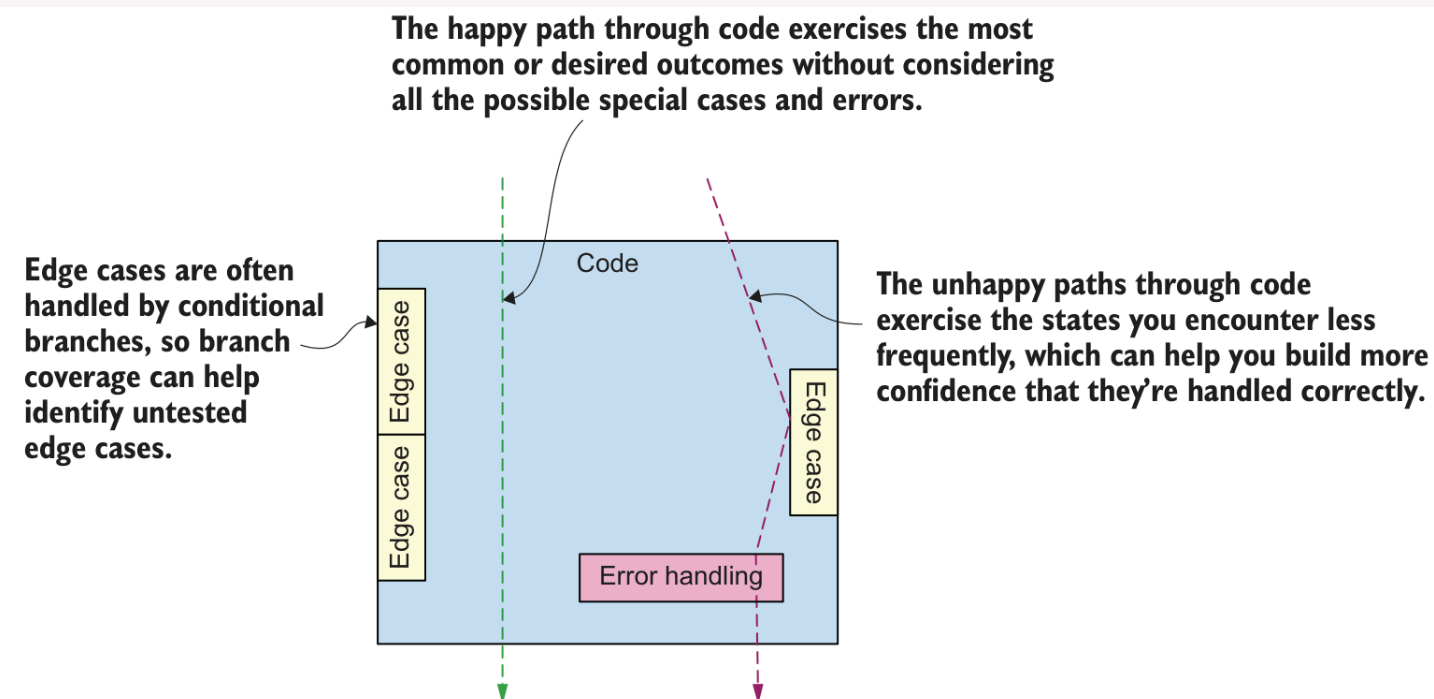


Figure 5.2 Tests may cover the common, desired execution paths or the less common edge and error cases.

같은 동작이 기대되는 입력끼리 구분

- 우리 예제의 인수 생략은 지원하는 정상 동작이다. 드물다는 이유만으로 오류라고 부르지 않는다.
- 공백 이름·여러 인수의 정책이 미정이면 먼저 계약을 정한다. 100% 결과만으로 모든 입력의 정확성을 보장할 수 없다.

대표 입력

기대 결과

Hee

Hello, Hee!

인수 없음

Hello, world!

공백 이름 " "

허용 정책 미정

16

공유 상태에 따른 테스트 실패

격리 진단용 tests/test_state_demo.py

- 인사 테스트에서는 `monkeypatch`가 복구했다. 이 전역 목록은 기본 pytest가 자동 복구하지 않는다.
- 두 테스트가 같은 `FRUITS`를 보므로 추가→길이 순서에서는 길이 2와 기대값 1이 충돌한다.

```
FRUITS = ["apple"]

def test_append():
    FRUITS.append("banana")
    assert FRUITS == ["apple", "banana"]

def test_len():
    assert len(FRUITS) == 1
```

교재 Figure 5.5: 순서를 바꾼 결과

- 파란 상자는 목록 상태, 초록 상자는 테스트다. 왼쪽은 길이→추가로 통과하고, 오른쪽 자홍 상자는 추가→길이에서 실패한다.

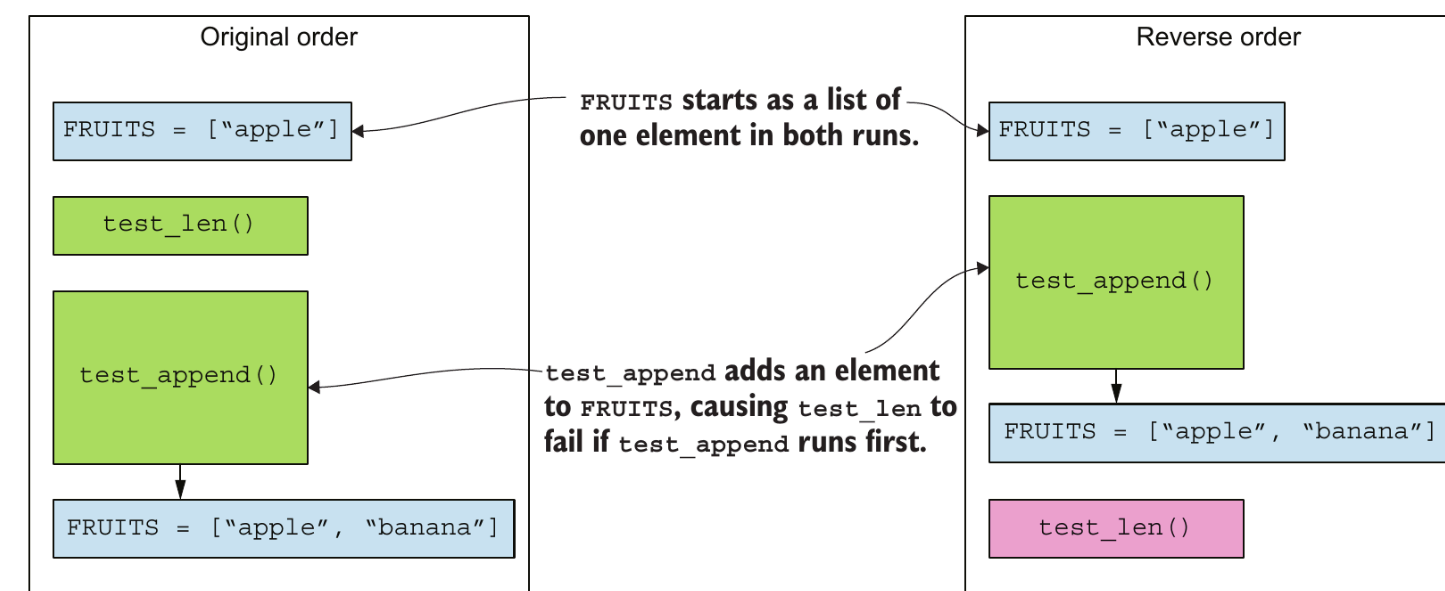


Figure 5.5 Tests that depend on state created by other tests may fail when reordered or moved.

추가 → 길이: 1 failed, 1 passed
 길이 → 추가: 2 passed (새 pytest 실행)

17

fixture를 이용한 상태 격리

tests/test_state_demo.py의 공유 상태 교체

- `@pytest.fixture` 는 준비 함수를 등록한다. 인자 `fruits` 로 요청하면 pytest가 반환값을 전달한다.
- 기본 function 범위에서는 테스트마다 준비한다. 이 함수가 매번 새 리스트를 만들기 때문에 두 테스트의 수정이 섞이지 않는다.
- 같은 전역 목록을 반환하거나 session 범위로 넓히면 상태를 공유할 수 있다. 아래 코드는 두 실행 순서 모두 `2 passed` 다.

```
import pytest

@pytest.fixture
def fruits():
    return ["apple"]

def test_append(fruits):
    fruits.append("banana")
    assert fruits == ["apple", "banana"]

def test_len(fruits):
    assert len(fruits) == 1
```

18

테스트 의존성과 설정

hello-open-source/pyproject.toml에 실행 방식 기록

- 명령으로 검증한 설정을 파일에 남기는 단계다. `[project.optional-dependencies]`의 `test` 키는 선택 설치할 도구 목록이다.
- `[tool.pytest.ini_options]`의 `addopts`는 기본 명령에 더할 인수다. 같은 표가 이미 있으면 키를 합치며 표를 중복 선언하지 않는다.
- `python -m pip install -e ".[test]"`는 현재 프로젝트와 test 그룹을 함께 설치한다. 일반 사용자는 이 그룹 없이 패키지를 사용할 수 있다.

```
[project.optional-dependencies]
test = ["pytest>=8,<10", "pytest-cov>=6,<8"]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q --cov=hello_open_source --cov-branch --cov-report=term-missing"
```

19

전체 검증과 실행 기록 전달

최종 인사 테스트 3개 확인

- 격리 진단 예제는 별도 실험이다. 최종 패키지는 `tests/test_greeting.py` 1개와 `tests/test_cli.py` 2개 사례다.
- `--collect-only` 는 수집만 확인한다. `--no-cov` 는 그 실행에서만 측정을 끄며 설정 파일은 바꾸지 않는다.
- 종료 0·3 passed·Miss 0·BrPart 0 을 확인한다. coverage 부족이 자동으로 테스트를 실패시키지는 않는다.

```
python -m pip install -e ".[test]"
python -m pytest --collect-only --no-cov
python -m pytest
```

```
3 tests collected → 3 passed
TOTAL: Stmts 10 / Miss 0 / Branch 2 / BrPart 0 / Cover 100%
```

README에는 환경·버전, 설치·검사 명령, 입력·기대값과 실제 결과를 남긴다. 발견 0개는 이름·경로, Import 오류는 Python·소스 경로, 단언 실패는 계약, coverage 공백은 Missing을 확인한다.

20

Summary

인사 패키지의 검증 결과와 판단

- 계약을 입력·기대값으로 고정하고 단언으로 확인했다. 함수 1개와 명령 입력 2개 사례가 모두 통과했다.
- 이름만 검사한 cli.py의 80%를 생략 사례 추가로 100%까지 채웠다. 실행 비율과 단언의 정확성은 서로 다른 증거다.
- 매개변수화는 같은 절차의 중복을 줄이고, 매번 새 상태를 만드는 fixture는 테스트 간 간섭을 막는다.